Ayudantía 4

Herencia y polimorfismo

Programación Avanzada | Sección: 03 | CIT 1010
Universidad Diego Portales

Contacto: isaias.vasquez1@mail.udp.cl Discord: _iszzza

Github: github.com/Isza285/Programacion-Avanzada-2026-01

Sección 1

Recordando modificadores de acceso

Modificadores de acceso

Modificador	Accesible en la misma clase	Accesible en las clases derivada	Accesible fuera de la clase
Public	Si	Si	Si
Protected	Si	Si	No
Private	Si	No	No

Sección 2

Herencia

Herencia

La Herencia permite heredar los atributos y métodos de una clase padre creada anteriormente

```
class Animal{
public:
    string nombre;
};

class Gato : public Animal{
public:
    void maullar(){
        cout << "meow" << endl;
    }
};
```

Acceso a atributos heredados

Cuando se crea una clase que hereda de otra tendremos:

- Acceder a atributos y métodos de la clase padre.
- Esto dependiendo del modificador:
  - Public y Protected: Es posible acceder
  - Private: No es posible acceder.

```
class Animal{
protected:
    string nombre;
};

class Gato : public Animal{
public:
    void mostrar(){
        cout << nombre << endl;
    }
};
```

Tipos de herencia

Clase base	Herencia public	Herencia protected	Herencia private
Public	public	protected	private
Protected	protected	protected	private
Private	No accesible	No accesible	No accesible

Ejemplo

```
class Animal{
public:
    string nombre;
protected:
    int edad;
};

class Perro : public Animal{
    // nombre (public)
    // edad (protected)
};

class Gato : private Animal{
    // nombre (private)
    // edad (private)
};
```

Importante: Al utilizar Se hereda todo (Atributos, métodos), pero no todo siempre es accesible

Sobreescritura de métodos

Una clase derivada puede redefinir un método de la clase padre para cambiar su comportamiento.

```
class Animal{
public:
    void hacerSonido(){
        cout << "Sonido generico" << endl;
    }
};

class Perro : public Animal{
public:
    void hacerSonido(){
        cout << "Guau" << endl;
    }
};
```

```
18 int main(){
19
20     Animal animal1;
21     Perro perro1;
22     animal1.hacerSonido();
23     perro1.hacerSonido();
```

```
Sonido generico
Guau
```

: Creando dos objetos de clase perro y animal y utilizando su método.

Problema al realizar sobreescritura

Al momento de utilizar punteros o referencias la sobreescritura no aplica de manera correcta.

El método que se ejecuta depende del tipo del puntero, no del objeto real.

Esto ocurre porque C++ decide que método ejecutar durante la compilación.

```
Perro* p = new Perro();
p->hacerSonido();
//Output:
//Guau (Sobreescritura correcta)
```

```
Animal* a = new Perro();
a->hacerSonido();
//Output:
//Sonido generico
//(Sobreescritura incorrecta)
```

Sección 3

Polimorfismo

Polimorfismo y el uso de virtual

El polimorfismo permite que un método de la clase padre tenga diferentes comportamientos, esto dependiendo del objeto de la clase hija que lo ejecute.

Para esto, se debe escribir **virtual** antes de la declaración del método. Esto dentro de la clase padre.

```
class Animal{
public:
    virtual void hacerSonido(){
        cout << "Sonido generico" << endl;
    }
};

class Perro : public Animal{
public:
    void hacerSonido(){
        cout << "Guau" << endl;
    }
};
//int main()
Animal* a = new Perro();
a->hacerSonido(); // Guau
```

Override

El **override** indicara que un método de una clase hija sobrescribe el método de una clase padre.

Nota: No es obligatorio el uso de override, pero nos asegura que la escritura de los métodos sean los correctos.

```
class Animal{
public:
    virtual void hacerSonido(){
        cout << "Sonido generico" << endl;
    }
};

class Perro : public Animal{
public:
    void hacerSonido() override{
        cout << "Guau" << endl;
    }
};
```

```
class Perro : public Animal{
public:
    void hacerSonidos() override{
        // error en el nombre
        cout << "Guau" << endl;
    }
};
```

Compilation failed due to following error(s).

```
main.cpp:13:10: error: 'void Perro::hacerSonidos()' marked 'override', but does not override
   13 |     void hacerSonidos() override{
      |          ^~~~~~~~~~~~
```

El compilador arrojara un error indicándonos que la clase que queremos sobrescribir no existe previamente.

Sección 4

Ejercitación

Ejercicio 1

Crear una clase *Vehículo* que tenga:

Atributo:

marca.

Metodos:

Un constructor que inicialice la marca.

Un método mostrarTipo() que imprima: Soy un vehículo de marca: [marca].

Crear una clase *Auto* que herede de *Vehículo*:

Debe tener un constructor que reciba la marca y la inicialice utilizando el constructor de la clase base.

Sobrescribir el método mostrarTipo() para que imprima: Soy un auto de marca: [marca].

En el main:

Crear un objeto de tipo Auto, inicializando la marca (por ejemplo: "Toyota").

Llamar al método mostrarTipo().

Ejercicio 2

Utilizando el código del ejercicio anterior, analizar el comportamiento del programa al utilizar punteros.

En el main:

1. Crear un puntero de tipo *Vehículo**
2. Asignarle un objeto de tipo *Auto* utilizando *new,* inicializando su marca(ej: “Toyota).
3. Llamar el método mostrarTipo() usando el puntero (usar ->)

El comportamiento esperable es que el método no se haya sobrescrito.

Solucion:

Utilizar *virtual* y *override* en los métodos de la clase padre e hija.

Comprobar nuevamente el funcionamiento del método en el main.

Ejercicio 3

Crear un programa que modele empleados usando herencia y polimorfismo.

Se pide:

- **Crear clase Empleado con**:
  - atributo nombre
  - método calcularPago() que imprime: Sueldo del empleado: ---
- Crear clases derivadas:
  - **EmpleadoTiempoCompleto**
    - atributo sueldoMensual
    - calcularPago() imprime el sueldo.
  - **EmpleadoPorHora**
    - atributos horasTrabajadas y pagoPorHora
    - calcularPago() imprime el pago total (horas × pago)

Ejercicio 3

En el main:

1. Crear un arreglo de *Empleado** de tamaño 2.
2. Asignar al arreglo lo siguiente:
   1. Un objeto de clase: *EmpleadoTiempoCompleto.*
   2. Un objeto de clase: *EmpleadoPorHora.*
3. Recorrer el arreglo y utilizar el método calcularPago().

Sección 5

Extra: Evaluaciones anteriores

Ejercicio 1 Control 2 2024-02

Se necesita crear un sistema de gestión de personas en la UDP.

Para llevar a cabo esto, usted debe crear una clase base llamada **Persona** con los atributos, **nombre** y **edad**, y un método virtual **mostrar_informacion()**.

Luego, debe crear las clases derivadas **Estudiante** y **Profesor**, con su propio método **mostrar_informacion()** para mostrar los detalles específicos de cada tipo de persona.

El **Estudiante** tendrá un **arreglo de cursos** y otro de **notas** (ambos de tamaño 5), y un método para **calcular el promedio de las notas**.

El **Profesor** tendrá un **arreglo de cursos impartidos** y un método para **agregar cursos**. Cree todos los métodos necesarios para cumplir con lo indicado.

Ejercicio 2 Control 2 2024-02

Cree un sistema de gestión para el transporte público, específicamente para las micros.

Cree la clase base **Micro** la cual debe tener atributos como **patente**, **capacidad**, y un **arreglo para almacenar los nombres de los pasajeros** (máximo 10 pasajeros). Implemente las clases derivadas **MicroUrbana** y **MicroInterurbana**. Agregue métodos para **subir y bajar pasajeros del bus**. Debe usar polimorfismo para mostrar la información de cada tipo de bus.

En resumen, usted debe tener:
- Clase Micro con un arreglo de nombres de pasajeros (máximo 10), además de los atributos, patente y capacidad.
- Métodos para subir/bajar pasajeros
- Método para mostrar la lista de pasajeros junto con los datos de la micro (debe indicar el tipo de micro).

Ejercicio 1 Laboratorio 2 2024-02

Un parque de la municipalidad de Santiago necesita de usted para crear un sistema de administración de sus zonas. Cada zona tiene un conjunto de recursos disponibles, como juegos (columpios, toboganes, sube y baja, etc.) en la **ZonaJuegos** o canchas (basquetbol, futbol, tenis, etc.) en el **AreaDeportiva.** El sistema debe permitir asignar a las personas (usuarios) a un recurso específico en cada área. Los usuarios solo pueden usar un recurso si está disponible. Implemente métodos para gestionar la **asignación** y **liberar** recursos, además de controlar el número de usuarios por recurso (máximo 2 usuarios por recurso). Además, usando polimorfismo, debe crear un método **mostrar_estado()** que muestre el estado de los recursos.
En resumen, usted debe tener:
- Clase base **AreaParque** con un arreglo de recursos (hasta 5 recursos) y una matriz para asignar hasta 2 usuarios por recurso.
- Clases derivadas **ZonaJuegos** y **AreaDeportiva**, donde cada una gestiona diferentes recursos.
- Métodos para **agregar recurso**, asignar un usuario a un recurso específico (si es que está disponible) y **liberar recursos**.
- Tener en cuenta que en la asignación debe asegurarse de que **no haya más de 2 usuarios por recurso**.
- Método **mostrar_estado()** que muestre el estado de los recursos de la zona (si están disponibles o no), **debe indicar además el nombre de la zona.**

Ejercicio 2 Laboratorio 2 2024-02

La farmacia ahumado necesita crear un programa de control de stock de medicamentos. Para esto, cree un programa donde exista una clase base **Medicamento** con atributos como nombre, precio, laboratorio y un método virtual **mostrar_info()**. Cree las clases derivadas **MedicamentoGenerico**, que cuenta con un atributo principio_activo y **MedicamentoDeMarca** que cuenta con un atributo tipo (si es pastilla, jarabe, etc.). Implemente además un sistema de control de stock usando para registrar cuántas unidades de cada medicamento hay disponibles, creando los métodos para aumentar y reducir el stock, debe tener en cuenta que cada medicamento tiene un stock inicial de 100 unidades.
En resumen, usted debe tener:

- Clase **Medicamento** con métodos virtuales para **mostrar información** y **gestionar el stock (aumento de stock y reducción de stock).**
- Clases **MedicamentoGenerico** y **MedicamentoDeMarca** que contenga el método mostrar información usando polimorfismo.
- **Constructores para inicializar las clases.**

Ejercicio 1 Solemne 1 2024-02

Como una medida preventiva de seguridad se ha dispuesto identificar 300 puntos críticos en la ciudad. Cada punto será vigilado mediante cámaras las 24 horas del día los 7 días de la semana. Cuando se detecta un delito se comunica inmediatamente a carabineros del plan cuadrante del lugar quienes actúan rápidamente. Terminado el evento carabineros informa el número de detenidos y la cantidad de personas que alcanzaron a fugarse (aproximado).

Se le ha solicitado su ayuda para los registros y estadísticas de esta medida preventiva. Al respecto, se le ha pedido que desarrolle un aplicativo en c++ utilizando orientación a objetos. Su ayuda se enmarca en el registro de los eventos y la posterior generación de estadísticas. Se le solicita que la solución contenga dos clases:

Ejercicio 1 Solemne 1 2024-02

- Evento, que tenga los siguientes atributos: número del punto crítico, cantidad de detenidos en el evento, cantidad de fugados en el evento.
- PuntosCríticos, que tenga como atributo un arreglo de 300 objetos tipo Evento, cada posición del evento representa un punto crítico específico.
- Por ahora, sólo se le pide lo siguiente:

a) Defina la estructura de las clases Evento y PuntosCríticos (sólo atributos)

b) Programe el constructor de la clase PuntosCríticos

c) Programe en la clase PuntosCríticos el método void registrarEvento(Evento *e), que opera de la siguiente manera:

i. Recibe como parámetro un objeto del tipo Evento

ii. Busca en el arreglo la posición del evento (número del punto crítico) y añade la información en la celda correspondiente (si la celda no contiene nada añade el objeto a la celda, si la celda ya tiene algo acumula en el objeto existente los valores recibidos)

Ejercicio 1 Solemne 1 2024-02

d) Programe en la clase PuntosCríticos el método void estadisticas(), el cuál debe imprimir las siguientes estadísticas

i. Porcentaje de puntos críticos sin novedad, vale decir donde no ocurrió ningún evento

ii. Porcentaje de puntos críticos con algún tipo de evento asociado

iii. Total de detenidos

iv. Total de fugados

v. Puntos críticos (indicar número) donde hay más fugados que detenidos

e) Programe un método main que permita agregar a un objeto de la clase PuntosCríticos la información de al menos 5 eventos, una vez ingresados dichos eventos se deberá imprimir las estadísticas correspondientes.

Para esta pregunta asuma que las clases Evento y Resumen cuentan con los métodos get's y set's.

Ejercicio 2 Solemne 1 2024-02

Se necesita programar una clase Triángulo con 3 atributos: lado a, lado b y lado c. Se necesita que la clase tenga las siguientes características.

a) Un constructor con 3 parámetros

b) Un método llamado bool validar() que indique si, de acuerdo a los valores de los lados, el triángulo es o no válido. Un triángulo es válido si la suma de dos de sus lados es siempre superior al tercer lado.

c) Un método llamado string obtieneTipo(), el que devuelve uno de estos tres tipos de triángulo: equilátero (3 lados iguales), isósceles (2 lados iguales y 1 distinto), escaleno (3 lados distintos). El tipo sólo se puede obtener si el triángulo es válido, en caso contrario debe devolver “triángulo inválido”.

Ejercicio 2 Solemne 1 2024-02

d) Un método llamado float obPerímetro() que calcula el perímetro del triángulo. El perímetro sólo se puede calcular si el triángulo es válido, en caso contrario debe devolver -1. Recuerde que el perímetro es la suma de sus lados.

e) Un método llamado float obArea() que calcula el área del triángulo. El área sólo se puede calcular si el triángulo es válido, en caso contrario debe devolver -1. Puede aplicar la fórmula de Herón:

$$A = \sqrt{s(s-a)(s-b)(s-c)}$$

Donde S = semiperímetro (perímetro / 2)

f) Programe en un main la instanciación y cálculo de área y perímetro de los tres tipos de triángulos de esta pregunta.